

Minor Assignment — 02

Game Programming with C++ (CSE 3545)

Department of Computer Science and Engineering, Institute of Technical Education & Research, SOA Deemed to be University

Q1. SFML header files, namespace, and Linux build commands

Header files: SFML is split into five modules, each with its own header. For a typical graphical game the following are used:

```
#include <SFML/Graphics.hpp>    // windowing + 2D graphics
#include <SFML/Window.hpp>      // (already pulled in by Graphics.hpp)
#include <SFML/System.hpp>      // (already pulled in by Graphics.hpp)
#include <SFML/Audio.hpp>       // sound / music
#include <SFML/Network.hpp>     // sockets (only if multiplayer)
```

Namespace: All SFML classes, enums and functions live inside the `sf` namespace. In our code we access them as `sf::RenderWindow`, `sf::Texture`, etc., or place using namespace `sf`; at the top of the file.

Linux compilation and execution:

```
# One-line compile + link
g++ main.cpp -o game -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio
```

```
# Two-step form (useful for multi-file projects)
g++ -c main.cpp
g++ main.o -o game -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio
```

```
# Execute
./game
```

***Note:** The link order matters — `-lsfml-graphics` must come before `-lsfml-window`, which in turn comes before `-lsfml-system`, because each module depends on the one after it.*

Q2. Declare four objects of class calculateArea

There are two equally acceptable C++ ways to create four instances of the class:

```
// Method 1: four individually named objects
calculateArea obj1, obj2, obj3, obj4;
```

```
// Method 2: an array of four objects
calculateArea objs[4];
```

***Note:** Both forms invoke the default constructor of `calculateArea` four times. Method 2 is preferred when the objects will be processed in a loop.*

Q3. Call setValues() and getValues() using an object

```
int main() {
    calculateArea rect;           // create an object
    rect.setValues(10, 20);      // call the mutator with length = 10, width = 20
    rect.getValues();             // call the accessor to display / return values
    return 0;
}
```

***Note:** Public member functions are invoked through the object using the dot operator `object.function(args)`. If the object were allocated on the heap (`calculateArea* p = new calculateArea;`) we would use the arrow operator: `p->setValues(10, 20);`.*

Q4. Open an SFML window of 960 x 540 pixels

```
#include <SFML/Graphics.hpp>
using namespace sf;

int main() {
    // VideoMode(width, height) -- width first, height second
    RenderWindow window(VideoMode(960, 540), "Game Window");
    return 0;
}
```

***Note:** sf::VideoMode stores the resolution, and RenderWindow's constructor uses it together with a title string to create the native OS window.*

Q5. Main game loop with Esc-to-quit (Timber!!!)

The game loop keeps running while the window is open. Inside it we (1) handle input, (2) update game state, and (3) draw the frame. Pressing Esc closes the window, which in turn terminates the loop on the next iteration.

```
while (window.isOpen()) {
    /* ----- 1. Handle the player's input ----- */
    Event event;
    while (window.pollEvent(event)) {
        if (event.type == Event::Closed)
            window.close();          // user clicked the X
    }

    if (Keyboard::isKeyPressed(Keyboard::Escape))
        window.close();             // user pressed Esc

    /* ----- 2. Update the scene ----- */
    // (move sprites, run physics, check collisions ...)

    /* ----- 3. Draw the scene ----- */
    window.clear();                  // wipe last frame
    // window.draw(spriteBackground);
    // window.draw(spriteTree);
    window.display();                // swap buffers
}
```

Q6. Draw soa.jpeg onto the window using Texture and Sprite

A Texture holds the pixel data in GPU memory; a Sprite is a lightweight object that references a texture and can be positioned / scaled / rotated independently.

```
#include <SFML/Graphics.hpp>
using namespace sf;

int main() {
    RenderWindow window(VideoMode(960, 540), "SOA");

    // 1. Load the image into a Texture
    Texture textureSOA;
    textureSOA.loadFromFile("soa.jpeg");

    // 2. Attach the texture to a Sprite
    Sprite spriteSOA;
    spriteSOA.setTexture(textureSOA);
    spriteSOA.setPosition(0, 0);    // top-left corner of the window

    while (window.isOpen()) {
```

```

    Event e;
    while (window.pollEvent(e))
        if (e.type == Event::Closed) window.close();

    window.clear();
    window.draw(spriteS0A);    // draw the image every frame
    window.display();
}
return 0;
}

```

Q7. Fly soa.jpeg from top-centre to bottom

The sprite's y-coordinate is increased by $\text{speed} \times \text{dt}$ on every frame so that its motion stays frame-rate independent. `setScale` shrinks the image so the whole flight is visible inside the window.

```

Texture textureS0A;
textureS0A.loadFromFile("soa.jpeg");

Sprite spriteS0A;
spriteS0A.setTexture(textureS0A);
spriteS0A.setScale(0.25f, 0.25f);    // scale factors (sx, sy)

// Start at top-centre of a 960 x 540 window
float spriteW = textureS0A.getSize().x * 0.25f;
spriteS0A.setPosition((960 - spriteW) / 2.f, 0.f);

bool flying = true;
float flightSpeed = 200.f;    // pixels / second
Clock clock;

while (window.isOpen()) {
    /* ... event handling & Esc check ... */
    Time dt = clock.restart();    // elapsed time per frame

    if (flying) {
        float dy = flightSpeed * dt.asSeconds();
        spriteS0A.move(0.f, dy);    // move DOWN the y-axis

        if (spriteS0A.getPosition().y > 540)
            spriteS0A.setPosition(    // loop back to the top
                (960 - spriteW) / 2.f, 0.f);
    }

    window.clear();
    window.draw(spriteS0A);
    window.display();
}

```

Note: SFML's coordinate system has the origin at the top-left, x increasing to the right and **y increasing downward**, so moving "down" means adding to y.

Q8. Fly soa.jpeg from left to right

Same pattern as Q7, but now the x-coordinate is updated instead of y. The sprite starts at $x = 0$, vertically centred, and is reset to the left edge when it exits the right side.

```

Texture textureS0A;
textureS0A.loadFromFile("soa.jpeg");

Sprite spriteS0A;
spriteS0A.setTexture(textureS0A);

```

```

spriteSOA.setScale(0.25f, 0.25f);

float spriteH = textureSOA.getSize().y * 0.25f;
spriteSOA.setPosition(0.f, (540 - spriteH) / 2.f); // vertically centred

bool flying      = true;
float flightSpeed = 200.f;                          // px / s
Clock clock;

while (window.isOpen()) {
    /* ... event handling & Esc check ... */
    Time dt = clock.restart();

    if (flying) {
        float dx = flightSpeed * dt.asSeconds();
        spriteSOA.move(dx, 0.f);                    // move RIGHT

        if (spriteSOA.getPosition().x > 960)
            spriteSOA.setPosition(0.f,
                                   (540 - spriteH) / 2.f);
    }

    window.clear();
    window.draw(spriteSOA);
    window.display();
}

```

Q9. Time-to-cross and bit-rate of spriteBee

Given: speed = 200 px/s width = 2000 px 1 px = 8 bits

(a) Time to cross the screen

$$t = \frac{\text{distance}}{\text{speed}} = \frac{2000 \text{ px}}{200 \text{ px / s}} = 10 \text{ seconds}$$

(b) Bit-rate

$$\begin{aligned}
 \text{bitrate} &= \text{speed} \times \text{bits-per-pixel} \\
 &= 200 \text{ px/s} \times 8 \text{ bits/px} \\
 &= 1600 \text{ bits/second} \\
 &= 1600 \text{ bps} = 1.6 \text{ kbps}
 \end{aligned}$$

***Note:** The bit-rate represents the effective stream of pixel information the bee "emits" while flying — a nice illustration of why pixels per second is also a bandwidth.*

Q10. Distance covered by spriteBee in one frame

`clock.restart()` simultaneously resets the clock *and returns* the time that elapsed since the previous restart. Let that value be `dt`. Because distance = speed × time:

```

Clock clock;
float beeSpeed = 200.f;          // pixels / second (given)

// inside the game loop, once per frame:
Time dt = clock.restart();
float dtSeconds = dt.asSeconds();

float distance = beeSpeed * dtSeconds; // pixels travelled this frame

```

Numeric example. If a frame takes 16 ms (roughly 60 fps):

$$dt = 0.016 \text{ s}$$

$\text{distance} = 200 \text{ px/s} \times 0.016 \text{ s} = 3.2 \text{ pixels per frame}$

***Note:** Multiplying by dt is what keeps motion consistent across machines — a faster PC renders more frames, each covering less distance, so the on-screen speed stays identical.*

Q12. Centered "SOA UNIVERSITY" text on a 1920 x 1080 screen

We load a custom font, build a Text object, set its character size and colour, and then re-centre its *origin* on its own bounding box so that setPosition places its middle at the middle of the window (not its top-left corner).

```
RenderWindow window(VideoMode(1920, 1080), "SOA University");

// 1. Load the font
Font font;
font.loadFromFile("KONIKAP.ttf");

// 2. Configure the text
Text messageText;
messageText.setFont(font);
messageText.setString("SOA UNIVERSITY");
messageText.setCharacterSize(100);
messageText.setFillColor(Color::Red);

// 3. Move the sprite's origin to the centre of its bounds
FloatRect tb = messageText.getLocalBounds();
messageText.setOrigin(tb.left + tb.width / 2.f,
                      tb.top + tb.height / 2.f);

// 4. Place that centre at the middle of the screen
messageText.setPosition(1920 / 2.f, 1080 / 2.f);

// in the game loop
window.clear();
window.draw(messageText);
window.display();
```

***Note:** Step 3 is the trick. By default a sprite's "anchor" is its top-left corner; resetting the origin to the bounding-box centre makes setPosition intuitively place the text middle.*

Q13. Red-filled rectangle (W x H) centred on a 1920 x 1080 screen

```
float X = 400.f, Y = 200.f;      // example width and height

RectangleShape rect(Vector2f(X, Y));
rect.setFillColor(Color::Red);

// Re-centre the origin so (X/2, Y/2) becomes the anchor
rect.setOrigin(X / 2.f, Y / 2.f);
rect.setPosition(1920 / 2.f, 1080 / 2.f);

// in the game loop
window.clear();
window.draw(rect);
window.display();
```

***Note:** Without setOrigin, the rectangle's top-left corner would land at the screen centre and the shape would drift down-right by (X/2, Y/2).*

Q14. Four green circles at the corners + one stretched red circle at the centre

Every corner circle is an independent `sf::CircleShape` of radius `X`, coloured green, positioned so the circle fits inside each corner. The central circle has its scale exaggerated so it "stretches" — here $2\times$ horizontally and $1\times$ vertically.

```
float X = 80.f;    // radius in pixels

// ----- four green corner circles -----
CircleShape topLeft(X),  topRight(X),
             botLeft(X),  botRight(X);

topLeft.setFillColor (Color::Green);
topRight.setFillColor(Color::Green);
botLeft.setFillColor (Color::Green);
botRight.setFillColor(Color::Green);

// positions are measured to the circle's top-left bounding corner,
// so a circle of radius X fills a 2X x 2X box
topLeft.setPosition (0.f,      0.f);
topRight.setPosition(1920 - 2*X, 0.f);
botLeft.setPosition (0.f,      1080 - 2*X);
botRight.setPosition(1920 - 2*X, 1080 - 2*X);

// ----- centre stretched red circle -----
CircleShape centre(X);
centre.setFillColor(Color::Red);
centre.setOrigin(X, X);           // centre-anchor the circle
centre.setScale(2.f, 1.f);        // stretch horizontally
centre.setPosition(1920 / 2.f, 1080 / 2.f);

// ----- draw -----
window.clear();
window.draw(topLeft);
window.draw(topRight);
window.draw(botLeft);
window.draw(botRight);
window.draw(centre);
window.display();
```

Note: `setScale(2.f, 1.f)` on a circle turns it into an ellipse — SFML simply multiplies the shape's vertices by the given factors on the GPU side.

— End of Assignment —